

- ✓ **Operating System:** a program that acts as an intermediary between a user and the computer hardware. Control programs and hardware by allocating resources among various apps and users for preventing errors (**protection** -> access to resource, permission), misuse of the computer (**security**), efficiency and fair resource use (**accounting**).
 - Execute user programs and make solving problems easier.
 - Make the computer system convenient to use.
 - Use the computer hardware in an efficient manner.
 - User interface.
 - I/O operations a running program may require I/O.
 - File system manipulation read and write files and directories.
 - **Communications** – Processes may exchange information.
 - **Error detection**
- ✓ Computer system structure: **hardware**, **OS**, **application** program (define ways in which the resources are used), and **user**.
- ✓ **Handheld** computers are resource poor, optimized for usability and battery life. Embedded computers do not have user interface.
- ✓ **Kernel:** a program running at all times on the computer.
- ✓ **Bootstrap program/firmware** is loaded at power-up or reboot, stored in ROM/EPROM and initializes all aspects of the system including operating system.
- ✓ **I/O** devices and the **CPU** can execute concurrently. **Devices controller**.
- ✓ A **trap** or **exception** is a software-generated interrupt caused either by a software error or a user request for OS service.
- ✓ **Pulling** and **vectored** interrupt system. **Program status:** register and program counter.
- ✓ **System call** (with c/c++ and accessed by API)– request to the OS to allow user to wait for I/O completion.
- ✓ **Types of system call**
 - Process control: create/terminate process, end, abort, load, execute, set/get, and wait.
 - File management: create, delete, open, close, read, write, print, rename, get, and set.
 - Device management: request, release, read, write, attach and detach devices.
 - Get/set system time, get/set system date, and get/set process/file.
 - Communication: create/delete communication connection, send/receive message.
- ✓ **System program**
 - Background program/ subsystem/services/ daemon: run in user context.
 - Program language support: compiler, debugger, assembler and so on.
 - Program loading and execution: absolute/ relocatable loader, linkage editor.
 - Application program
- ✓ **Device-status table** contains entry for each I/O device indicating its type, address and state.

- ✓ Storage: **main-memory, secondary storage, hard disk, and solid state disks.**
- ✓ Caching, devices driver (provide interface b/n kernel and controller). <1.18>
- ✓ **Direct memory access:** device controller transfer blocks of data without CPU intervention.
- ✓ How modern computer works: CPU, devices, and instruction and data.
- ✓ **Multiprocessor:** asymmetric (each processor is assigned to special task) and symmetric (each processor performs all tasks). Economy scale, Increase reliability and throughput.
- ✓ Clustered system multiple systems work together and share storage via storage area ntw.
- ✓ A job selected and run via **Job scheduling** (batch system), **timesharing (multitasking** -> user program and tasks), **CPU scheduling, swapping, virtual memory, dual mode (user mode and kernel mode =0), mode bit.** <1.28>
- ✓ **Timer** is set to prevent infinite loop.
- ✓ Computer environment: **traditional, mobile, distributed, client-server (CSS & FSS), peer to peer, virtualization trough VMM (emulation), cloud computing** (public, private, hybrid) like software, platform, and infrastructure, **and real-time embedded system.**
- ✓ **Command line interpreter:** implemented in kernel or by system program.
- ✓ Most users' view of the operation system is **defined by system programs**, not the actual system calls.
- ✓ **Registry:** used to store and retrieve configuration information.
- ✓ UNIX consists of two separable parts (Systems programs and the kernel).
- ✓ **Mach** example of **microkernel:** easier to extend, reliable, secure, and overhead.
- ✓ **Process:** a program (passive form of process) in execution that progress in sequential fashion. (Text section, heap, stack, program counter, and data section are parts of process).
- ✓ Process states and its diagram: **new, running, waiting, ready, and terminated.**
- ✓ Process control block (**PCB**): information associated with each process. Like PC & IO status.
- ✓ Scheduling queue: **job queue, ready queue, and device queue.**
- ✓ **Short term scheduler** (CPU scheduler): decides which process should be executed next.
- ✓ **Long term scheduler** (job scheduler): decides which process should be brought into ready queue.
- ✓ **Two types of processes:** IO bound and CPU bound process.
- ✓ **Medium term scheduler** is added to decrease the degree of multiple programming.
- ✓ Multitasking in mobile phone: **a foreground process and multiple background process.**
- ✓ **Context switch:** the system saves the state of old process and loads the saved state for the new process.
- ✓ **Process termination with abort() system call:**
 - Child exceeded allocated resource.
 - Task assigned to the child is no longer required.
 - Parent is exiting.

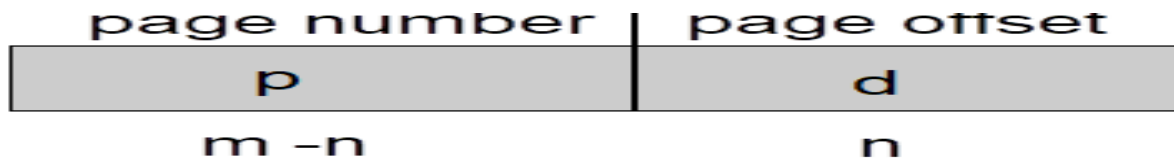
- ✓ Process is **zombie** when no parent is waiting, and is **orphan** when parent is terminated without invoking wait.
- ✓ **Inter-process Communication** through message passing (processes synchronize their action through communication link) or shared memory (communication is under the control of the users processes not the operating system). Independent and cooperating (info sharing, communication speedup and modularity).
- ✓ **Threads**: multiple program counter per process. They don't share stack as well as register.
- ✓ Process creation is heavy-weight while **thread creation is light-weight** because it can access shared data.
- ✓ Benefits of thread: **responsiveness, resource sharing, economy** (can interact without the need of inter-process communication), and **scalability**.
- ✓ **Multi-core vs. multiprocessor, parallelism** (data/task) **vs. concurrency** (support more than a task to make progress on single core).
- ✓ **Many-to-one**: when user-level threads mapped to a single kernel thread.
- ✓ **One-to-one**: when each user-level thread maps to kernel thread.
- ✓ **Many-to-many**: allows many user level threads to be mapped to many kernel threads. Like Windows with the *ThreadFiber* package.
- ✓ **Two-level mode**: allow user thread to be bound with kernel thread. Like IRIX & HP-UX.
- ✓ Critical section problem: **entry** section, **critical** section, **exit** section and **remainder** section.
- ✓ Solution to Critical-Section Problem: **mutual exclusion, progress, and bound waiting**.
- ✓ **Critical-Section Handling in OS: preemptive**: allow preemption of the process when running in the kernel model. **Non-preemptive**: runs until exits kernel mode.
- ✓ **Peterson's Solution**: only for two processes. Load and store as atomic instruction. The three requirements are met.
- ✓ **Synchronization Hardware**: based on idea of locking. Set and test instruction/ compare and swap instruction. **Mutex Locks/ binary semaphore**: acquire and release function.
- ✓ **Semaphore**: `wait(S) {while (S <= 0); S--;}` and `signal(s){ s++;}`. Counting semaphore.
- ✓ Applications may spend lots of time in critical sections and therefore **Semaphore** is not a good solution.
- ✓ **Dispatcher**: switching context, switching to user mode, and jump into proper loc.
- ✓ **Scheduling Criteria**: CPU utilization, through put, turnaround time, waiting time, and response time: amount of time it takes from when a request was submitted until the first response is produced.
- ✓ **Convoy effect**: short process behind long process.
- ✓ **FCFS Scheduling, Shortest-Job-First Scheduling, Shortest-remaining-time-first, Priority Scheduling, round robin**.
- ✓ Multilevel Queue: **foreground** (RR) and **background** (FCFS) **2.4(6.20)**

- ✓ **Deadlock:** two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes.
 - ✓ **Deadlock Characterization:** mutual exclusion, hold and wait, no preemption, and circular wait.
 - ✓ **Claim edge, request edge, and assignment edge.**
 - ✓ Single instance: resource allocation graph. Multiple instances: use banker's algorithm.
 - ✓ Handling Deadlocks: **prevention, avoidance, and recovery** (process termination depends on resource and time and preemption depends on cost).
 - ✓ **Priority Inversion:** scheduling problem when lower-priority process holds a lock needed by higher-priority process.
-

Chapter three

- ✓ Program into memory => process. Main memory and registers (access in one CPU clock or less) only accessed by CPU. Cache found in between.
- ✓ **Memory unit:** sees physical addresses + read or addresses + data and write requests.
- ✓ Base/relocatable and limit registers define the logical address space. Used to protect user processes from each other, and from changing OS code and data.
- ✓ Addresses binding: Symbolic (source code addresses) => relocatable addresses (compiled code – 14) => absolute addresses (linker/loader – 74014).
- ✓ Binding of instructions and data to memory: compile time (absolute code), load time (relocatable code if location is not known at compile time) and execution time (when there is reference made to location in memory).
- ✓ Logical/virtual addresses consist of **segment number and offset** that are generated by CPU.
- ✓ Logical and physical addresses differ in execution time address binding scheme.
- ✓ Memory management unit (**MMU**) a device maps virtual address to physical address.
- ✓ **Dynamic relocation using a relocation register:** it increases CPU utilization and throughput with no increase in response time or turnaround time.
 - Since unused routine is never loaded, there is better memory-space utilization.
 - Useful to handle large amounts of code and error code.
- ✓ Static linking and dynamic linking (postponed until execution time).
- ✓ Swapping (roll out and roll in) in backing store: occurs when total physical memory space of processes exceed physical memory and higher-priority process is in the ready queue.
- ✓ **Double buffering:** transfer I/O to kernel space, then to I/O device.
- ✓ Standard swapping and **modified version** (swap only when free memory extremely low).
- ✓ Swapping on mobile system not supported instead throw out read-only data or terminate apps:
 - Due to small amount of space, poor throughput and limited number of write cycles.
- ✓ Main memory has two partitions: resident OS held in low memory with interrupt vector and user processes held in high memory.
- ✓ Degree of multiprogramming is limited by the number of partitions. Variable partition for efficiency purpose. Adjacent free partitions is always combined.
- ✓ Dynamic storage allocation problem: **first-fit**, **best-fit** (produces the smallest leftover hole, and **worst-fit** (allocate the largest hole).
- ✓ **External and internal fragmentations.**
- ✓ **Compaction:** shuffling memory contents at execution time when relocation is dynamic.
- ✓ Segments: main program, procedure, function, method, object, variable, block, stack, etc.

- ✓ **STBR** points to the segment table's location in memory. **STLR** indicates number of segments used by a program.
- ✓ Code sharing occurs at segment level and segments vary in length.
- ✓ **Paging**: when physical address space is not contiguous.
- ✓ **Frame**: divided physical memory into fixed-size blocks. **Paging** divided logical memory into same size blocks.



- ✓ 2^m logical space address & 2^n page size. $n = 2$ & $m = 4$ 32 byte memory and 4 byte pages.
- ✓ Worst case fragmentation is 1 frame – 1byte & average case fragmentation is $\frac{1}{2}$ frame size.
- ✓ **PTBR** points to the page table and **PTLR** indicates size of the page table.
- ✓ Associative memory/**translation look-aside buffers** is fast-lookup hardware cache.
- ✓ Memory protection **implemented by associating protection bit**.
- ✓ **Structure of the page table**: hierarchical, hashed (Common in address spaces > 32 bits contains virtual page number, value of the mapped page frame, and a pointer to the next element), and inverted page table (instead of tracking logical pages, track all physical pages decreases memory needed but increases time needed).
- ✓ **Virtual memory**: allows address spaces to be shared by several processes and more programs running concurrently.
- ✓ **Virtual address space** logical view of how process is stored in memory.
- ✓ Invalid reference => abort and not-in-memory => bring to memory. Swapper that deals with pages is a **pager**. I => page fault.
- ✓ Extreme case in demanding paging: start process with no pages in memory known as **pure demand paging**.
- ✓ Trap=> save state=> was the interrupt was a page fault=> was the reference legal=>
- ✓ **Copy-on write** allows both parent and child processes to initially share the same pages in memory.
- ✓ **Page replacement** when there is no free frame. Used to prevent over allocation of memory and let's only modified pages written to disk.
- ✓ How many frames to give each process and which frames to replace (**frame allocation algorithm**), want lowest page fault rate on both first access and re-access (**page replacement algorithm** like FIFO and least recently used).
 1. FIFO => Belady's anomaly adding more frames can cause more page faults.
 2. Optimal algorithm: Replace page that will not be used for longest period of time
 3. Least recently used: Replace page that has not been used in the most amount of time. Implemented with counter using clock time or with stack (when the page is referenced move it to the top).

4. LRU approximation algorithms: with **reference bit**. When the page is referenced bit is set to 1. **Second chance (clock) page replacement algorithm**: if page to be replaced has reference bit is 0 then replace it. If reference bit is 1 then set its reference bit to 0, leave page in memory and replace next page considering the above rules.
 5. (0, 0) best page to replace -> (0, 1) not recently used but modified used by **enhanced second chance algorithm**.
 6. Counting algorithms: least frequently used and most frequently.
- ✓ **Thrashing**: a process is busy with swapping pages in and out. Multiprogramming might cause thrashing. When total size of locality > total memory size, thrashing may occur.
 - ✓ **Transfer rate, positioning time (random access/seek time), and rotational latency**.
 - ✓ **Access latency** = average access time = average seek time + average latency (60/RPM * 0.5).
 - ✓ **Average I/O time** = average access time + (amount to transfer / transfer rate) + controller overhead.
 - ✓ Solid state disks: busses can be slow, but no seek time/rotational latency.
 - ✓ Magnetic tape: used for backup and storage of infrequently used data.
 - ✓ Logical block is the smallest unit of transfer.
 - ✓ Disk **bandwidth** = bytes transferred/ total time.
 - ✓ **Disk scheduling**: FCFS, SSTF (a natural appeal, default), SCAN/elevator algorithm, and C-LOOK.
 - ✓ LOOK (default) a version of SCAN, C-LOOK a version of C-SCAN. Arm only goes as far as the last request in each direction.

Chapter 4

- ✓ **File**: text, source, executable, regular, directory, character, block, symbolic, pipe, socket files. Is known by its index number/descriptor not by its name.
- ✓ **File operations**: create (creating directory entry), write, read, delete, truncate, open, and close.
- ✓ **File structure**: none, simple, and complex structure.
- ✓ **Directory** is a collection of nodes containing information about all files.
- ✓ USF and ZFS are general file system.
- ✓ **Directory operation**: search, list a directory, traverse the file system, etc.
- ✓ **Directory organization**: based on efficiency, naming, and grouping. Like single level directory, two level directory, **tree structured** directories, and acyclic graph directory (have shared subdirectories and files).
- ✓ **Link** is a pointer to an existing file (two types hard and soft/short links). Use back pointer to prevent dangling pointer.
- ✓ **NFS**: is a common distributed file sharing method. None user data: Meta data.

- ✓ **File sharing:** manually via programs like FTP, automatically like DFS, and semi automatically via WWW. Stateless protocols are less secure.
- ✓ **Sector:** is the smallest unit. File does not know its name and current size.
- ✓ **Layered file system:** application program (passes file name) -> logical file system (take the file name) -> file organization module (convert logical address to physical address) -> basic file system (compute basic operation like read/write) -> I/O control (execute the device driver functionality) -> devices.
- ✓ **Buffers** hold data in transit. Mounting point (my computer) .Vs mount point (root for flash)
- ✓ File control block/ index node hold information about file.
- ✓ **Boot control block** contains info needed to boot OS from that volume.
- ✓ **Cooked/raw:** just a sequence of blocks with no file system. Root partition contains the OS.
- ✓ **Virtual file system:** allows same system call interface (API) for different types of file systems
- ✓ File type does not make a difference for OS. App cares about extension.
- ✓ **Directory implementation:** **linear** list (simple to program but time consuming to execute) and **hash** table good if entries are fixed size/ use chained- overflow method.
- ✓ Efficiency depends on **disk allocation** (contiguous (performance is high), linked, FAT (file allocation table), and indexed allocation (file has its index block/s of pointers to its data blocks)).
- ✓ Performance depends on **file access type**. Keeping data and metadata close together reduce seek time.
- ✓ Free space management with bit map but requires extra space or with linked space list.
- ✓ Write is faster than read due to random access.
- ✓ **Recovery:** consistency checking by comparing data in directory structure with data blocks on disk and tries to fix inconsistencies.
- ✓ **Log structure file system:** record each metadata update to the file system as a transaction.
- ✓ **Sun network file system:** allows accessing of remote files across LANs/WANs in a transparent manner.
- ✓ **NFS:** it operates in a heterogeneous environment of different machines, OS, and network architectures through RPC (remote procedure call) -> search, read, manipulating links, etc. NFS servers are stateless and do not use the concept of caching and concurrency-control.
- ✓ **Three major layers of NFS:** UNIX fs interface -> virtual fs -> NFS service layer.
- ✓ **WAFL:** optimized random I/O and write. Root inode -> inode file (free block, free inode, and file). An important change from more standard file systems is that the free-block map has more than one bit per block.

Chapter 5

- ✓ **I/O** devices like storage, transmission, and human interface. And they have data in, data out, status, and control registers.
- ✓ **Port:** connection point for devices.

- ✓ **Polling:** read busy bit, set by controller from status register until 0. Inefficient.
- ✓ **Interrupts** is the good way of making processor efficient. Interrupt-request line is triggered by I/O device and check by processor after each instruction. Also used for exceptions.
- ✓ **Maskable** to ignore/delay some interrupts.
- ✓ When vector number is multiply by 40 hexadecimal, physical address that handle the interrupt.
- ✓ **Direct memory access** through DMA controller to avoid programmed I/O for large data movement.
- ✓ **Device driver layer** hides differences among I/O controllers from kernel.
- ✓ **Devices vary** in character stream vs block, sequential vs random, synchronous vs asynchronous, sharable vs dedicated, speed of operation, and read-write, read only, or write only.
- ✓ **Kernel I/O structure:** kernel -> kernel I/O subsystem -> device driver -> controller -> devices.
- ✓ In blocking process suspended until I/O completed yet in asynchronous process runs while I/O executes. Non-blocking implemented via multi-threading.
- ✓ **Vectored I/O:** allows one system call to perfume multiple I/O operations to decrease context switching and system call overhead.
- ✓ **Spooling:** if the device can serve only one request at a time, will hold output for a device like printing.
- ✓ To protect I/O devices **I/O must be performed via system calls.**
- ✓ To improve performance in I/O reduce number of context switches, data copying, interrupt, use DMA, and smarter hardware devices.

Chapter six

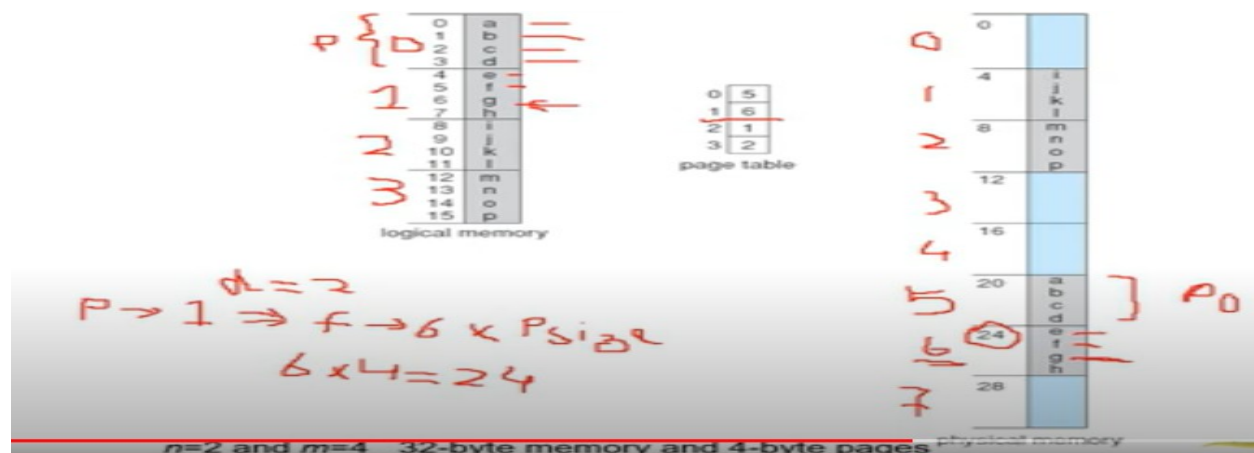
- ✓ **Principles of protection:** principle of least privilege.
- ✓ **Domain switching** via passwords (su) or commands (sudo).
- ✓ Copy and owner privilege is applicable to an object while control is applicable to domain object.
- ✓ **Implementation of access Matrix:**
 - Global table (won't fit in main memory).
 - Access control list defines who can perform what operation or what operations allowed on what object.
 - Capability list instead of object-based, list is domain based (capability is representation of object by its name/ address).
 - Lock-key (object has lock and domain has key).
- ✓ **Revocation of access rights:** immediate, delayed, selective, general, partial, total, temporary, and permanent.

- ✓ **Security violation categories:** breach of confidentiality, breach of integrity, breach of availability, theft of service, denial of service (prevention of legitimate use).
- ✓ **Security violation methods:** masquerading (authentication), replay attack, man in the middle attack, and session hijacking.
- ✓ **Program threats:** Trojan horse like spyware, trap door detected by inspection of code, logic bomb, stack and buffer overflow, viruses (very specific CPU architecture, operating system, applications).
- ✓ **Keystroke logger:** to grab passwords, credit card numbers
- ✓ **Network threats:** worms, port scanning,

Answers:

1. Putting the critical section of producer and consumer inside the monitor -> if the buffer is full/empty the process will be blocked/woken with wait/signal primitive.
2. Allows multiple processes to read and write data to the message queue without being connected to each other, much easier to implement, tolerant of higher communication latencies
3. Deadlock happens when multiple processes are waiting in the ready queue for an event(s) to occur that should be initialized by one of them while starvation occurs when a process waiting to be assigned to a processor for indefinite period of time. Deadlock in multi-programmed system.
4. No because shortest remaining time first.
5. A. 9,13,10,11. B.

Q 8.2 line 4, 8.15 line 1, 8.48

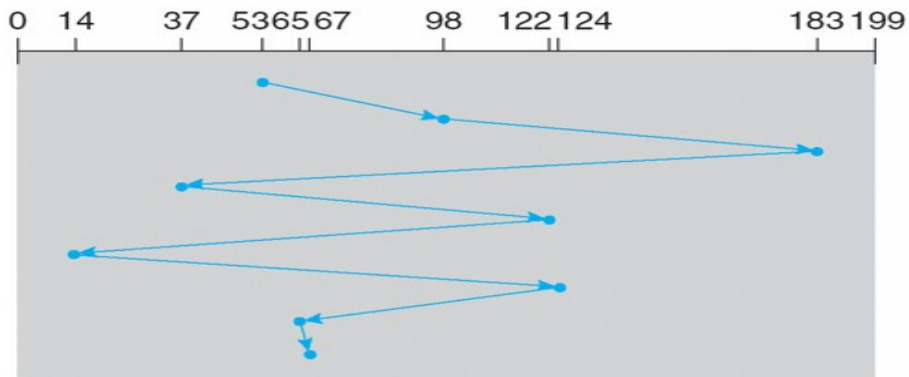


FCFS

Illustration shows total head movement of 640 cylinders

queue = 98, 183, 37, 122, 14, 124, 65, 67

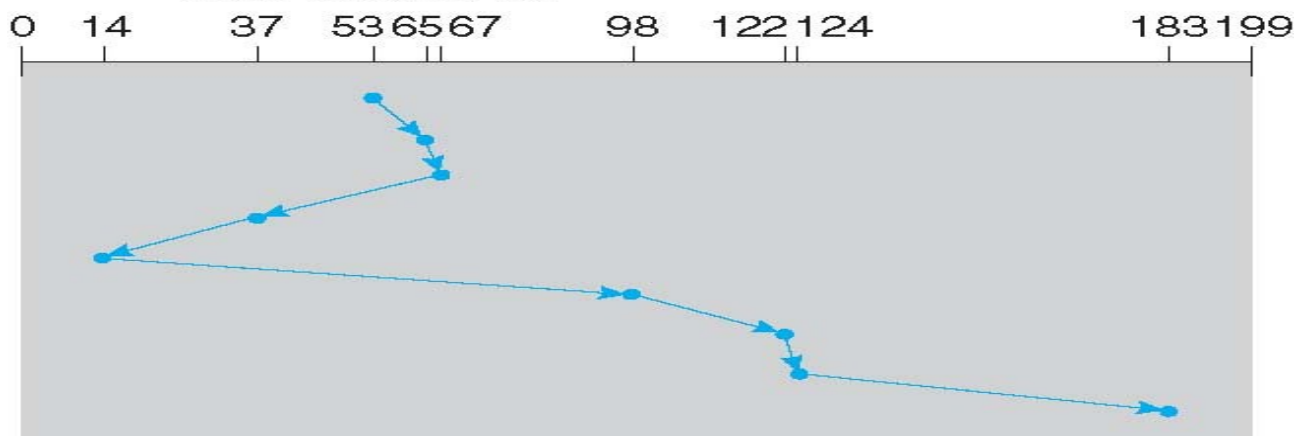
head starts at 53



SSTF

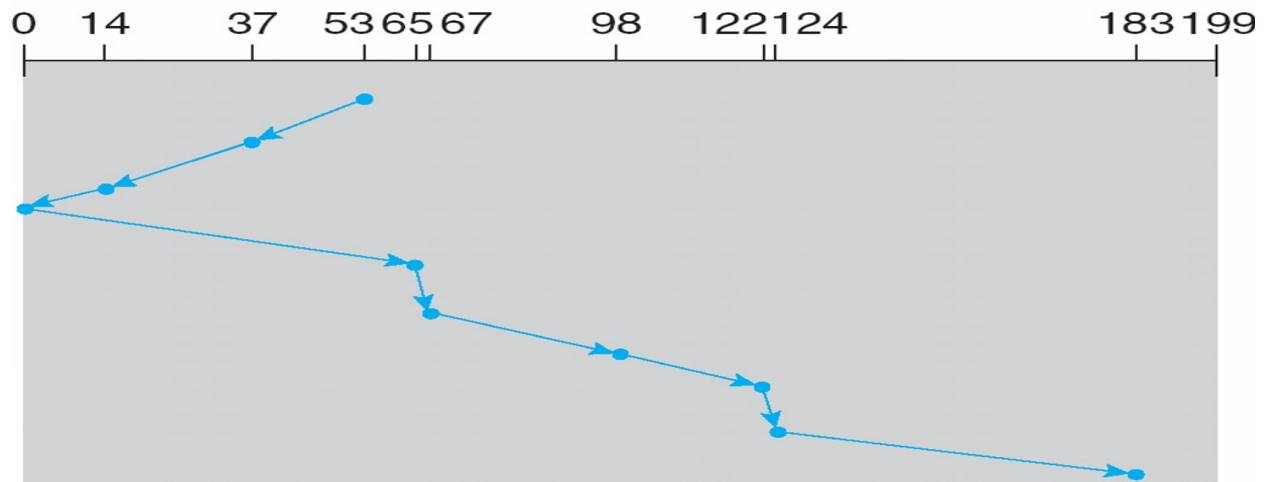
queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



SCAN

queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53



C-SCAN

queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53

